

10. Web Services

Overview

*By the end of this chapter, you will be able to identify key factors in selecting a third-party web service; explain the basic concepts of a RESTful web service; determine the correct headers to add to a request; explain common web service authentication and authorization schemes; create and read request bodies in JSON; perform manual API testing using a REST client; and compose **GET** and **POST** requests in PHP using Guzzle and then process the results.*

This chapter presents the basic concepts of web services and explains how to connect your application with them using Guzzle, a popular PHP open source library for making HTTP requests.

Introduction

In the previous chapter, we learned how to use PHP's package manager, Composer, to include third-party packages in your application. By doing so, you saw how to benefit from the open source solutions to problems that have already been solved and drastically reduce the amount of code you must produce and maintain in your own projects.

Web services are a technology that is enabling a lot of innovation in our industry. There are countless web services available on the internet, with some requiring a paid account to access their service and some freely available to the public as long as you don't surpass a rate limit. This is important because it means you don't have to own all the data you use in your app. You can leverage the data and systems others have built and then build on top of them, stringing them together to provide functionality that is unique to your application. PHP is a language built specifically for the web in the age of web APIs. By some, it has been called the "best glue" to piece together a collection of external services.

In this chapter, we will present an overview of web services and show you some examples of interacting with them. If you are unfamiliar with what a web service is, the term is generally used to refer to an application service that is either publicly accessible or available within an intranet that can be programmatically interacted with to retrieve or alter data. In other words, a web service is a server or cluster of servers that is accessible via a network and that processes requests generated by computer processes as opposed to a user entering a URL into a browser. Some of the most well-known web services are public APIs exposed by social networks, such as Facebook or Twitter, which allow authorized applications to gain access to their user's data. An e-commerce application might use a FedEx web service for verification of the shipping address on an order before accepting it. Another basic example is a large database of movie data that allows clients to look up data related to a specific title, actor, or director.

HTTP is the protocol used by these services in order to communicate. It is the same protocol a web browser uses to request a web page from a server. Making a request to a web service uses the same request/response cycle you learned about in *Chapter 6, Using HTTP*, and, in fact, you can make some requests directly from your browser.

An Example Web Service

As a quick example, we can use a site that we will be interacting with later in the chapter using PHP, but for now, let's see what happens when you browse to <https://packt.live/33iQi0M>. This is a simple web service that receives the request from the client, reads the public IP address of the network that the request is coming from, and sends a response containing that IP address back to the client in a format that computers and humans can both easily read. Here is a screenshot of what you would see in your browser; however, note that the IP address would be different because it is dependent on your actual location:

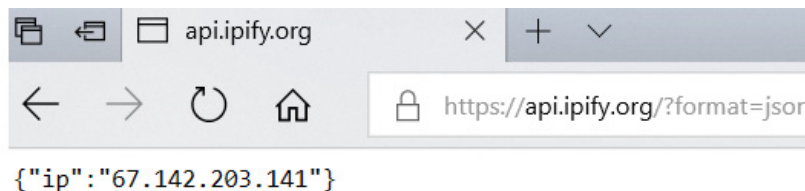


Figure 10.1: Printing the IP address

This is a very simple service, but it illustrates the concepts we are trying to learn without the need for very complicated business logic. When you enter the preceding URL into your browser, you should see some curly braces, colons, and double quotes formatted around some text. The text should indicate the public IP address of the network your computer is on. Your browser makes an HTTP **GET** request to the server, then the server processes your request and returns a formatted response back to your browser. PHP has tools to make these requests programmatically and then parse the results so that they are usable by your applications.

Selecting Third-Party APIs

Sometimes, you will not have a choice of which web service to integrate your application with, either because it is the only service that provides the functionality you need or because some other constraint has limited your options. When this is not the case, it is useful to have a set of guidelines you can use to compare web services against each other to aid in your selection, for example, a business contract obligation. Some of the things you may want to consider (in no particular order) are documentation, stability, availability, and pricing.

If you've ever integrated with a third-party API before, you know the value of having clear, concise, and complete documentation to lead you through the process, as opposed to the difficulty of there being an absence of quality documentation. Without complete documentation, you will find yourself at the mercy of a support chain that may be slow to respond, if there is even support available at all. Be sure to read through the documentation of any API and understand it before committing it as a dependency for your application.

The stability of the web service you choose is also another consideration to bear in mind. If you are paying for the service, you may be able to get a guarantee of uptime in a **service-level agreement (SLA)**. This won't always be the case, and you might not have reliable data on the stability of a third-party system, but there are other things you can inquire about, such as how they handle system maintenance and rolling out new versions of the API.

Availability has a number of different meanings in this context. In some cases, performance will be of utmost importance to your application. In those cases, if you depend on live calls to external systems, you will want to ensure the web service is available to respond to your requests in a timely fashion. A performant web service will return responses measured in microseconds rather than seconds. Another aspect of availability is that some web services may limit the number of requests you can make to their service in a given timeframe, for example, the number of requests Facebook accepts per hour. If this is the case, you will need to ensure that the web service will support the number of requests your application is likely to make during peak usage. Of course, if the data you are working with is cacheable, then that is always a preferable option.

Some web services are available to use free of charge or simply with the creation of an account, but some require paid access. Often, if there is a charge to use a web service, they will have a pricing model that uses a tiered structure, allowing a specified number of requests per billing period. If the price is too high for your business model to support, this may eliminate some services as options.

RESTful Concepts

Many of the web services you see these days will identify themselves as RESTful web services. This is an important concept to know, both for interacting with these services and designing your own. **Representational State Transfer (REST)** is a style of developing an API, rather than a protocol such as HTTP or **Simple Object Access Protocol (SOAP)**. It is a set of design constraints for architecting a web service and was first defined by Roy Fielding in his Ph.D. dissertation, *Architectural Styles and the Design of Network-based Software Architectures*.

Rather than try to cover the full dissertation, we will cover some of the important concepts you will need to know for interacting with RESTful services. The first is that they are stateless. This means that each request to the server happens in isolation. In other words, the server should not need any knowledge of previous requests by the client in order to process the current request. Each request should contain all the information necessary to process that request. This provides the benefits of simplicity, scalability, and flexibility in the API.

The next concept is that RESTful APIs expose their functionality by representing resources through the URLs that are requested. Each URL represents a single resource or collection of resources, and the HTTP method (**GET**, **POST**, **PUT**, **PATCH**, or **DELETE**) you use to make the request determines whether you are retrieving the resource, creating it, updating it, or deleting it, respectively. The URL will be the same for all those operations; only the HTTP verb changes. The URL for a specific resource will also contain a unique identifier for that resource. Let's say there was a fictitious web service located at **acme.com/api** and one of the resources you can interact with through the API was called **products**. To retrieve a record with an identifier of **123**, you would make a **GET** request to **api.acme.com/products/123**. To update that record, you would make a **POST** request to **api.acme.com/products/123** with a **POST** body that would contain a representation of the product to be updated. Similar requests could also be made to create and delete records. The **api.acme.com/products** URL would give you a listing of products. The combination of URL and HTTP verb is known as **endpoint**, which is a very common term in RESTful APIs literature.

As a consumer of these APIs, you will want to pay attention to the HTTP status codes to determine the success of your request. These are standardized codes that give information about the response from the server. These codes are divided into five groups: 1xx, 2xx, 3xx, 4xx, and 5xx. We have seen a definition of these in *Chapter 6, Using HTTP*. You can see the full list with explanations of what the values represent at <https://packt.live/2M2NfnH>.

For a **GET** request to retrieve a resource, a status code of 200 represents success. A request to create a record would return a status code of 201. If you request a resource that does not exist, you would expect to get back a status code of 404. These are the most common status codes, and it's a good idea to be familiar with them.

Another quality of RESTful APIs is that the responses should define whether or not they are cacheable, that is, whether they are fit to be stored on the client side for a period of time to avoid making a duplicate request. Some requests won't be cacheable, such as requests to update data, or resources that are updated frequently. For any request that is cacheable, it is likely to be in your best interest as a consumer of the API to cache it if you are going to request it frequently. This helps reduce the total number of external requests your application makes, which can increase your performance dramatically.

The last concept you should be familiar with is known as **Hypermedia As The Engine Of Application State (HATEOAS)**. This principle states that a client should be able to dynamically navigate through the application using hypermedia links contained in the response content.

The simplest example of this is when responding to a PUT request to create a new resource; a hypermedia link to the resource (**acme.com/api/widgets/123**, in our earlier example) is returned as metadata in the response. While this is one of the architectural constraints that makes a web service fully REST compliant, many do not apply it due to the extra effort required to complete this stage. However, it is important to be aware of it as you may come across it in the future.

Request Formats

There are two main formats you will use to format the data you send to servers in your requests: XML and JSON. Both of these provide hierarchical structures for formatting data so that it may be easily read by both computers and humans.

Extensible Markup Language (XML) has been around since 1998, extending from its predecessor, Standard Generalized Markup Language, which is a standard for how to specify a document markup language. If you are familiar with HTML, the markup language interpreted by web browsers to make web pages, XML will seem very similar. You can think of XML and HTML as cousins. Just as in HTML, data elements in XML are wrapped in sets of opening and closing tags, each beginning with the **<** symbol and ending with the **>** symbol. The closing tag is signified with a backslash preceding the element name. A full example of an XML element with data inside would look like this: **<element>Some Data Here</element>**. These element tags can be nested as well, creating a nested hierarchy. For each level of nesting, the text is indented for readability.

Here is an example:

```

<element>
    <property attr="some attribute">value</property>
    <items>
        <item>some value</item>
        <item>some other value</item>
    </items>
</element>

```

Each element can also have attributes, which are placed inside the opening tag, like so: **<element attribute="some value">**. This gives XML a lot of flexibility in modeling data structures, allowing for space to store metadata without affecting the rest of the structure. However, this is a trade-off, paying for the flexibility with complexity and verbosity. These downsides are part of the reason why much of the web community has begun shifting to a newer, more concise format named JSON.

JSON is an abbreviation of **JavaScript Simple Object Notation**. Despite having JavaScript as part of its name, JSON is a language-independent data format. JSON is independent now, but it was invented as a way to express Objects in JavaScript, and it was popularized as a data transfer support to avoid XML, which is heavier, and more expensive, to transfer through Internet. JSON uses curly braces to wrap data objects, double quotes to indicate properties and string values, and square brackets to wrap arrays. Commas separate items in a sequence, which can be properties or array items. Items are indented to keep things organized, just as in XML. This structure should give a sufficient visual representation:

```

{
    "property": "value",
    "some array": [
        "item 1": "some value",
        "item 2": "some other value"
    ]
}

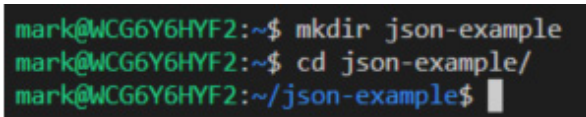
```

JSON is great because it's concise, which makes it fast over the wire. It also allows for easy conversion from objects to JSON strings and back to objects in memory. PHP offers two built-in functions that take care of these processes for you, **json_encode** and **json_decode**. With **json_encode**, you pass in the object that you want to transform into JSON and it will return it, while **json_decode** does the opposite. It's worth noting, if you are decoding JSON into objects, you will get objects of the generic **stdClass** type instead of the original type prior to encoding. JSON does miss out on the descriptiveness provided by XML, and therefore you may see metadata represented in the properties of the objects. However, in general, it is easier to read, easier to write, and less complex to interact with in code.

Exercise 10.1: JSON Encoding

In this exercise, we will prepare some data for a fictitious email marketing web service that allows you to add your data through their API so that you can send out emails through their platform to your mailing list. If it was a RESTful web service, it would likely accept a PUT request with a body in JSON format at an endpoint such as **/recipient**. The purpose of this exercise is to simply demonstrate translating PHP objects into JSON, and we will cover actually sending the requests later on in the chapter:

1. Create a new folder for this example, **json-example**, and navigate to the folder through the Terminal, as follows:



```
mark@WCG6Y6HYF2:~$ mkdir json-example
mark@WCG6Y6HYF2:~$ cd json-example/
mark@WCG6Y6HYF2:~/json-example$
```

Figure 10.2: Navigating to the desired folder

2. Create a **MailingListRecipient** class in a PHP file with the same name. Include public properties for **\$email**, **\$firstName**, and **\$lastName**, which are passed in through the constructor:

```
<?php
class MailingListRecipient
{
    public $email;
    public $firstName;
    public $lastName;

    public function __construct($email, $firstName, $lastName)
    {
        $this->email = $email;
        $this->firstName = $firstName;
        $this->lastName = $lastName;
    }
}
```

3. Create a file called **json.php** that requires the **MailingListRecipient** class:

```
<?php
require 'MailingListRecipient.php';
```

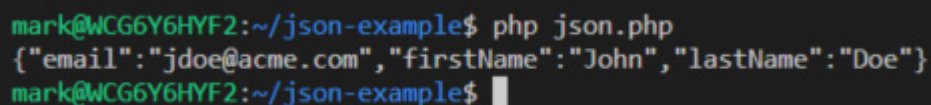
4. Instantiate a new **MailingListRecipient** class:

```
$recipient = new MailingListRecipient('jdoe@acme.com', 'John', 'Doe');
```

5. Encode the recipient variable as a JSON string and write it to the output:

```
$requestBody = json_encode($recipient);  
echo $requestBody.PHP_EOL;
```

6. Run the script to see the string as JSON that is ready to be sent as a request body:



```
mark@WCG6Y6HYF2:~/json-example$ php json.php  
{"email":"jdoe@acme.com","firstName":"John","lastName":"Doe"}  
mark@WCG6Y6HYF2:~/json-example$
```

Figure 10.3: Displaying the string as JSON

When you are integrating with a web service as the client, the body of your request will need to be formatted to match the content type specified in your headers in the request. Some web services support multiple request/response data formats, allowing you to request the format that suits you best, while others will require you to use a specific format.

HTTP Headers

Every HTTP request and response is sent with a number of headers that facilitate communication between the client and server or provide meta-information about itself. Some headers will be automatically generated for you as part of the client making the request, such as **Host**, **User-Agent**, or **Content-Length**. It is important to be familiar with the extra headers you might want to include when making a request, as they can give you some control over the response you receive or the headers that might even be required for the request to be accepted.

The first of these is the **Accept** header. It allows you to specify a comma-separated list of content types expressed as a MIME type, such as **text/html** or **application/json**, which will be used to negotiate with the server to determine a mutual response body so that the client can correctly parse the request. The client may provide multiple content types it will accept, and the server will select one and specify which content type was used to format the response in the response headers. If the client is sending a POST request with a body, the **Content-Type** header should be provided to assist the server in parsing the data being sent. Most commonly, you will see this passed as **application/json** or **application/xml**.

The **Cache-Control** header in a server response will give information as to whether the response can be cached for later use by the client. This is typically only done for responses to GET requests but is nevertheless useful for decreasing the total number of requests made to a service if you are using data that is of a cacheable nature, thereby improving the performance of the application. If a response is cacheable, it will have a **max-age** header that specifies the number of seconds in which a request should be kept before it can be considered invalid and a new request generated.

If a request needs to be authenticated, the client may be required to pass an **Authorization** header. We cover authentication and authorization in the next section.

Authentication and Authorization

As a good security practice, web servers are designed to verify the user's identity and authenticate that the requested resource is accessible to the user. It is important to recognize the distinction between these two terms. Authentication is the process of validating that the user is who they say they are. This may be done as simply as checking a password or API key against one stored on the user's account, or it may be as complex as hashing values that contain a "secret" value known only by the client and the server. It has become common practice these days to have a separate authentication server to handle this duty, and by doing so taking that responsibility off the application server and handling it in a centralized manner.

Authorization is the process of verifying that an authenticated user has access to the resource they are requesting, whether it's viewing data or altering it. For example, if a service has basic-level access provided to anyone with a free account, but also provides a member subscription service where only certain endpoints are available to paying members, it would need to verify the authenticated user has permission to protected resources when they are requested. Another use case for authorization would be when users are only given access to resources they have created themselves, or can read any created resources, but can only edit their own.

We will take a moment to give a brief overview of some of the common authentication and authorization schemes you may run into. The first is open authentication, meaning the web service does not verify the user's identity. This is not very common, because it is not very secure. Still, there are some cases where it is acceptable, such as some of the example services that we will make use of later in the chapter.

Next is authentication by API key, where a user has created an account with the web service and requested a key that will be included as part of each request. This functions in a similar way to a username and password login process on a website, where you provide an account ID and API key, and the web service verifies the API key belongs to your account before processing your request. This is significantly more secure than open authentication, and most of the public web services you interface with will use this method.

Finally, there is the combination of Open ID Connect for authentication and OAuth 2.0 for access authorization. These are separate protocols that work together to provide a complete access control solution. Open ID Connect was built on top of OAuth 2.0 to shore up the security holes that were left by services using only OAuth 2.0 as a pseudo-authentication mechanism. In short, the client authenticates through an Open ID server, which may be a well-known internet company such as Google, Facebook, Microsoft, or Twitter, or it may be a company's internal authorization provider. After authenticating, a token is provided back to the application, which can then use it to make a request to the resource server. If we do end up integrating with one of these services, we can use the PHP league's Composer package for OAuth, which can be found on GitHub at <https://packt.live/35s7tiv>.

Manual API Testing

Sometimes, while you are integrating with a new web service, you will need to go through a process of trial and error to get your requests formatted in such a way that the service will accept it. In these cases, it can be hugely beneficial to have a client that will allow you to manually construct requests, send them to the service from the client, and display the response, allowing you to eliminate your own code from being the source of the problem.

Once you get a successful response, you can recreate the request correctly in code. Sometimes, this is a necessary step in troubleshooting and, at the very least, it can save you lots of frustration when trying to debug the code. I'll describe a few of the options available to you in the next few paragraphs.

If you prefer to have a client directly in your IDE to reduce the number of applications you have open during development, some have a REST client directly integrated into them. Jet Brains' PhpStorm IDE has an integrated client that, in many ways, works similarly to Insomnia. PhpStorm is a great IDE filled with countless beneficial features that speed up development, but it is a licensed software product and requires a subscription. If you have the means, it is definitely worth the cost.

If you are just sending GET requests, these clients may seem like overkill, but if you are sending a POST request with a body or need to send custom headers for authentication, clients like these might be your only option to manually test the web service. If you are going to be integrating with web services, it's well worth it to set up one of these clients.

The client we will use here for manual web service testing is called Insomnia, and can be found at <https://packt.live/2VuRco8>. It is a thick client that you'll have to install to use, but it has a nice intuitive interface that makes it simple to compose requests of all types and easily see the results.

Exercise 10.2: Manual API Testing with Insomnia

In this exercise, we will demonstrate using Insomnia to manually make a web service request to the **ipify** endpoint that we called through the browser at the beginning of this chapter. The benefit of using a client like this as opposed to a browser is that you can set request headers or form data that you would not be able to set from a browser:

1. Open Insomnia and click on the **New Request** button. Then, enter **Ipify** as the request name:

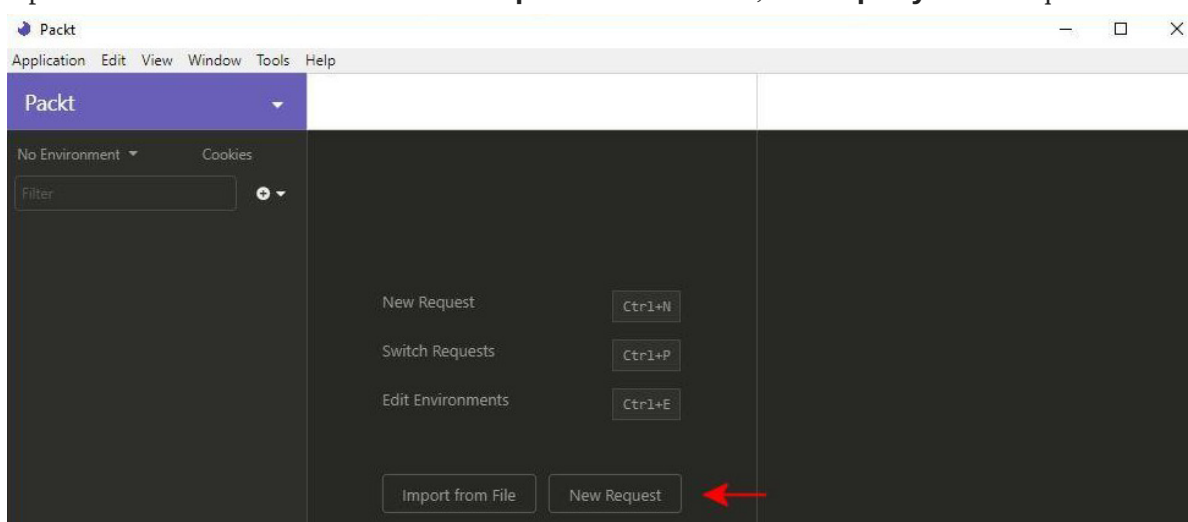


Figure 10.4: The Insomnia interface

2. Ensure that the request method is set to **GET**:

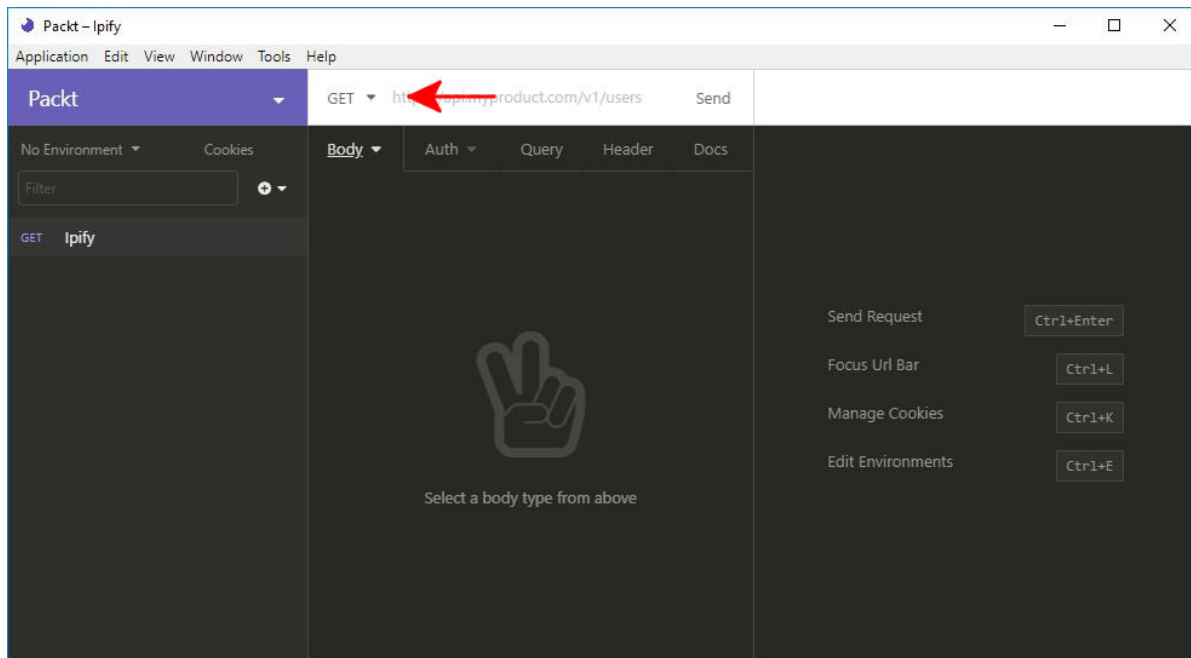


Figure 10.5: Checking the request method

3. Enter <https://packt.live/2oyJqxB> in the URL bar:

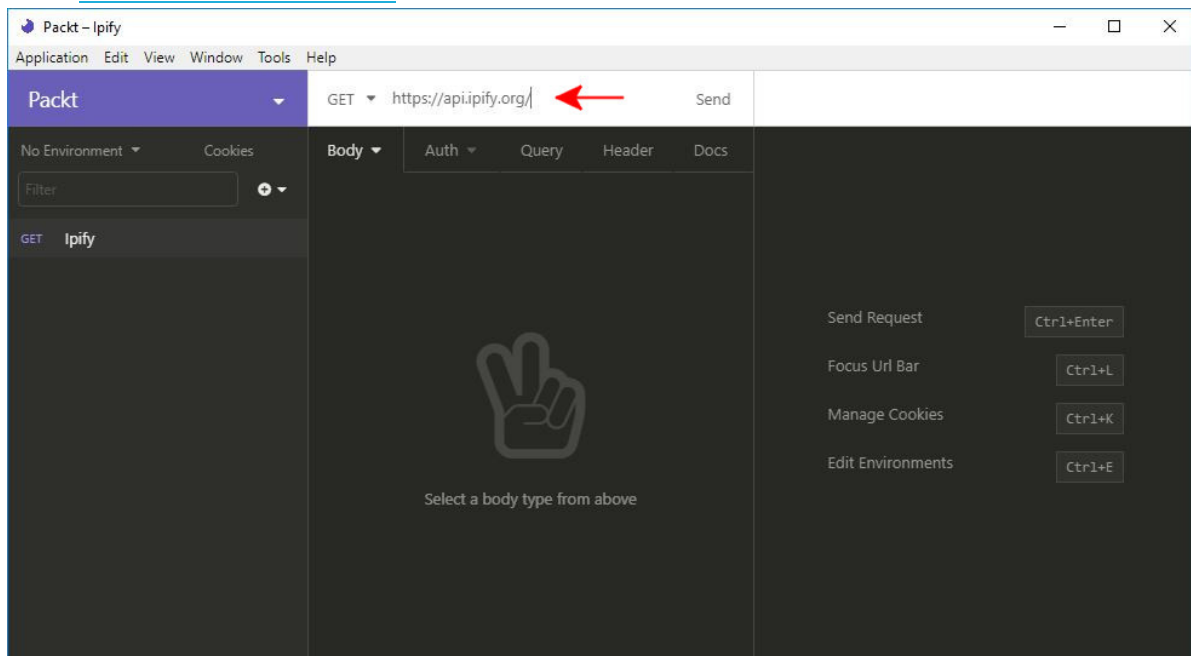


Figure 10.6: Adding the URL

4. Open the **Query** tab and enter **format** into the first new name field and **json** into the first new value field:

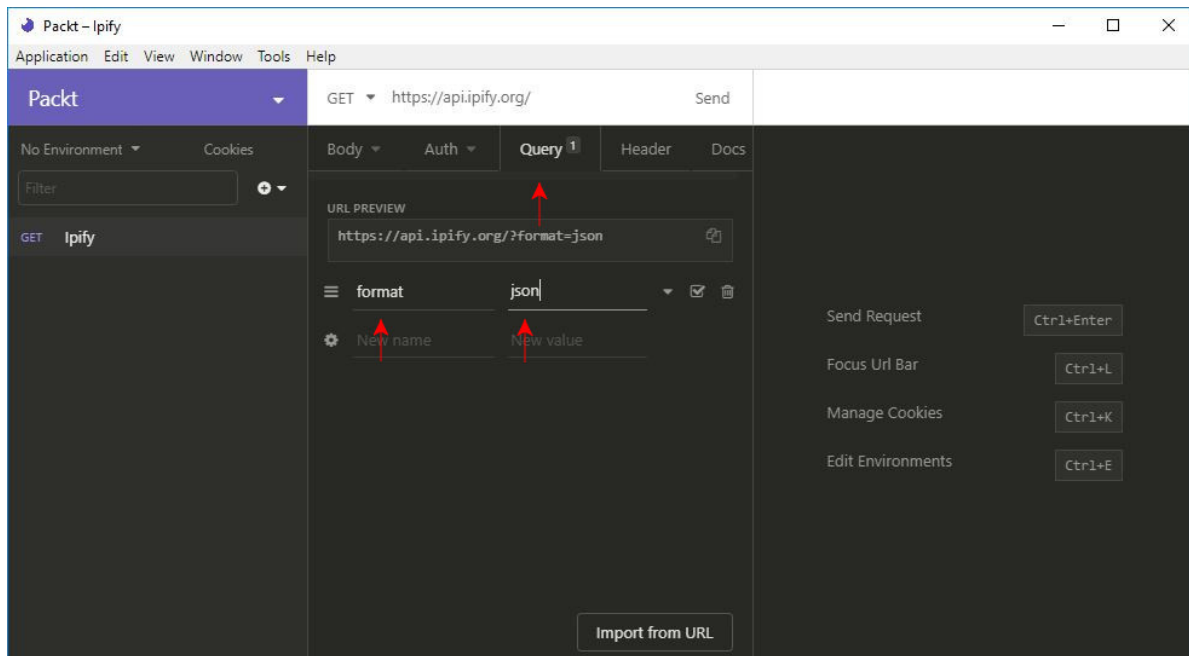


Figure 10.7: Adding data in the Query tab

5. Click on the **Send** button at the end of the URL bar:

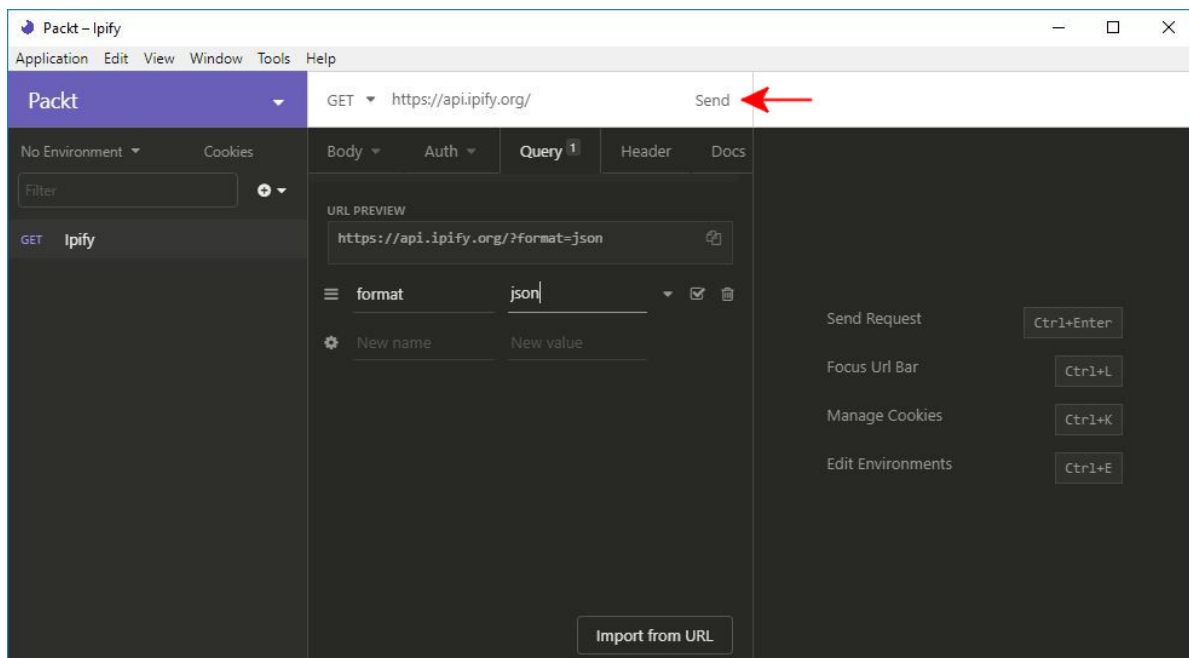


Figure 10.8: Sending the URL

6. You will see a **Preview** section displaying the JSON response:

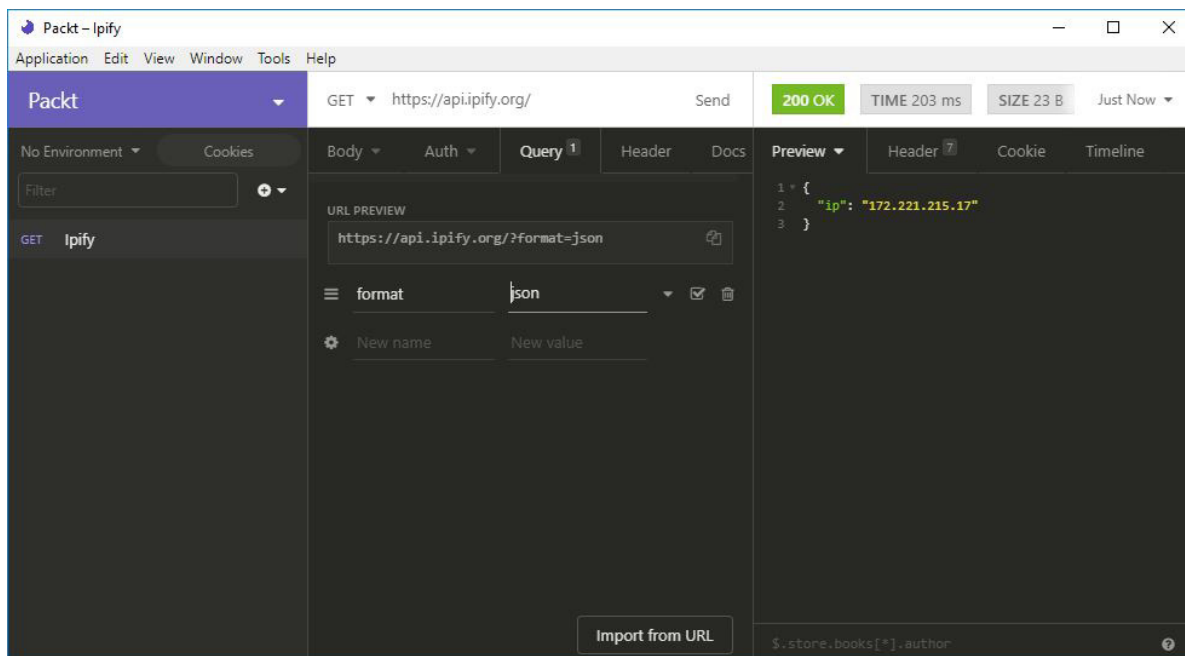


Figure 10.9: JSON response

Making a Request with PHP

Now that we've got all that theory out of the way, we get to cover actually making a request in PHP. There are several approaches you can use to make requests in the language, and ultimately all of them end up using the cURL extension to make the request. If you had a simple GET request to make, then you could use the built-in **file_get_contents** function. You could use the cURL functions to interact with the cURL extension directly, which are well documented at <https://packt.live/2olkmKv>; however, this can be tedious and is lacking a level of abstraction that can be provided by an object-oriented approach. For this, there is a package provided by Composer called **guzzlehttp/guzzle**. Guzzle is actually the official implementation of the PSR-7 standard for an HTTP message interface and is widely used.

Exercise 10.3: Making a GET Request with Guzzle

In this exercise, we will go over the process of instantiating a Guzzle client, configuring the request, and calling the method to send a **GET** request:

1. First, create a new project for this chapter in the directory where you keep your code and change into that directory:

```
mkdir guzzle-example
```

2. Initialize a new Composer project (refer to *Chapter 9, Composer*, if you need help) and then install Guzzle:

```
composer init
composer require guzzlehttp/guzzle
```

3. Create a PHP script named **ipify.php** and require the Composer autoload file:

```
<?php
```

```
require 'vendor/autoload.php';
```

4. Reference the **GuzzleHttp\Client** class with the **use** statement, and then instantiate a new **Client** object, passing in the base URL of the **ipify** web service:

```
use GuzzleHttp\Client;

$client = new Client(['base_uri'=>'https://api.ipify.org']);
```

5. Make an HTTP **GET** request to the root of the web service, passing in a format query parameter with a value of **json**, and store it in a **\$response** variable:

```
$response = $client->request('GET', '/'), ['query'=>['format'=>'json']]);
```

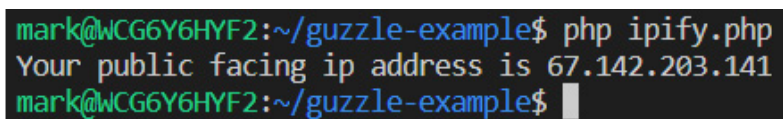
6. Extract the body of the response, which is a JSON string, using the **getBody()** and **getContents()** methods, pass the string through the **json_decode()** function to parse it into an object, and then store it in a **\$responseObject** variable:

```
$responseObject = json_decode($response->getBody()->getContent());
```

7. Echo a string to print out the **ip** property of the response object:

```
echo "Your public facing ip address is {$responseObject->ip}".PHP_EOL;
```

8. Run the script from the command line. You should see output similar to the following screenshot:



```
mark@WCG6Y6HYF2:~/guzzle-example$ php ipify.php
Your public facing ip address is 67.142.203.141
mark@WCG6Y6HYF2:~/guzzle-example$
```

Figure 10.10: Printing the IP address

Let me walk through the example code line by line to explain what it is doing. First, we are including the Composer autoload file so that all of our dependencies are automatically included as we covered in the previous chapter. Then, we add a **use** statement so that we don't have to refer to the full path of **GuzzleHttp\Client** every time we want to reference it. Then, we instantiate an instance of the **Guzzle** client, setting our target web service base URL in the **options** array passed into the constructor. Next, we call the request method on the **Guzzle** client. This accepts the HTTP method as the first parameter, which, in this case, is a **GET** request. The second parameter is the relative URI of the resource we are trying to access, which, in this case, is just the root, so we just enter a backslash. The final parameter is an array of options, which we populate with an associative array that tells the web service we would like the body of the response to be formatted in JSON.

After we have our response, we call the methods provided by **Guzzle** in a chain to get the body object, and then to get the contents of the response as a string, which will be JSON formatted text in this example. To be able to access the data in the response, we pass it through **json_decode** to turn it into a generic **stdClass** object that allows us to access the properties. Finally, we echo out a string to the output using string interpolation to inject the **ip** address returned by the service into our message.

Note

It is possible to decode JSON string into an array instead of an object.

Sending a **GET** request is useful, and many of the requests you write will use this method, but we should also cover sending a **POST** request, where you will have to provide some data to the web service to be processed. We have found another simple free web service that will allow us to make such a request, and you also might find it useful. It is a service that allows you to pass an email address and some options in a JSON string in the body of your request and it returns a **SpamAssassin** score for that email address. We will also demonstrate setting **Accept** and **Content-Type** headers in the request to tell the web service how to parse our request body and what format we will accept the response in. It is important to check your API calls for error conditions, and we will show some examples of this as well.

Exercise 10.4: Sending a POST Request with Headers

This exercise will be similar to the previous one, with the main difference being that we will be using the **POST** method to send data in the body of the request. This time, the service we are calling is one that accepts an email address and returns the **SpamAssassin** score for that email. **SpamAssassin** is an open source project by the Apache Software Foundation that helps system administrators filter emails from sources that send unsolicited bulk emails:

1. Create a **spamcheck.php** script in the same folder as the previous chapter. Require the Composer autoload file, add a statement to use the **GuzzleClient** class, and define a variable with any email address as a string:

```
<?php
require 'vendor/autoload.php';
use GuzzleHttp\Client;
$email = 'test@test.com';
```

2. Instantiate the **GuzzleClient** object, passing the URL of the service in the constructor:

```
$client = new client(['base_uri'=>'https://spamcheck.postmarkapp.com/']);
```

3. Create an array for the body of our request with the first item being the email variable and having a key of **email** and the second item being the string **short** with a key of **options**. Then, transform it into a JSON string using the **json_encode()** function and store it in a **\$requestBody** variable:

```
$requestBody = json_encode(['email'=>$email, 'options'=>'short']);
```

4. Open a **try...catch** block and, inside it, make a **POST** request to the **/filter** endpoint. The **Accept** and **Content-Type** headers are included in the **options** array as well as our request body:

```
try
{
    $response = $client->request('POST', '/filter'), [
        'headers' => [
```

```

        'Accept'=>'application/json'
        'Content-Type'=>'application/json'
    ],
    'body'=>$requestBody
]);

```

5. Check the HTTP status code of the response and, if it is not **200** (that is, successful), throw an exception:

```

if($response->getStatusCode()!==200){
    throw new Exception("Status code was {$response-
>getStatusCode()}, not 200");
}

```

6. Parse the JSON string response into an object and store it in a variable:

```

$responseObject = json_decode($response->getBody()->getContents());

```

7. If the **success** property is not set to true on the **response** object, throw an exception:

```

if($responseObject->success!== true){
    throw new Exception("Service returned an unsuccessful response:
{$responseObject->message}");
}

```

8. Output a string that states the **SpamAssassin** score for the email:

```

echo "The SpamAssassin score for email {$email} is {$responseObject-
>score} ".PHP_EOL;

```

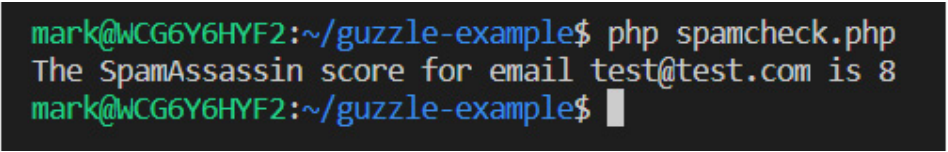
9. Catch any exceptions that may have been thrown and output the message:

```

catch
{
    echo "An error occurred: ".$ex->getMessage().PHP_EOL;
}

```

10. Run the script and see the output:



```

mark@WCG6Y6HYF2:~/guzzle-examples$ php spamcheck.php
The SpamAssassin score for email test@test.com is 8
mark@WCG6Y6HYF2:~/guzzle-examples$

```

Figure 10.11: The final output

This example is similar to the last one in many ways, with a few exceptions. First, we create a JSON string for the body of the request using **json_encode** to transform an associative array and store it in the **\$json** variable.

When we make the web service call using the request method, we pass **POST** as the HTTP method, and this time the relative path is **/filter**, making the full requested URL <https://packt.live/3269n6i>. In the **options** array, we include a headers array containing key-value pairs for the headers we want to include in our request.

The **Content-Type** header tells the web service that our body is formatted as JSON, and the **Accept** header tells the service we are expecting the format of the response to be JSON. If you needed to include other headers in your request, you could do this by adding them to the array. The **\$json** variable containing our JSON string for the payload of our request is passed in the **body** parameter.

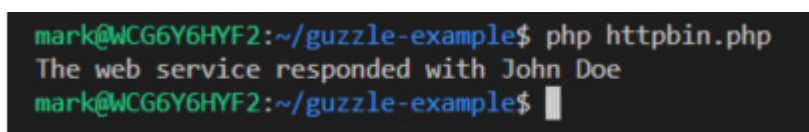
This time, before we get the content out of the response, we check to make sure we have a valid response. In most cases, the easiest way to do this is to look at the HTTP status code. Successful responses will be in the 2xx range. Most of the time, you will be able to look for 200 or 201, depending on which HTTP method you are using. After decoding the response, we check to make sure the **success** property is set to true. This is another layer telling us the request was processed correctly. Not all web services will provide this layer in the same way, but it is fairly common to include some indicator in the body of the response. If we find a condition indicating the request was not successful, we throw an exception with a message clearly indicating what failed, and handle it in the **catch** clause, passing the message onto the user.

Activity 10.1: Making Your Own POST Request to httpbin.org

Now it's time to practice making a request on your own. To do this, you will use a different service located at <https://packt.live/2oyJqxB>. **Httpbin** is an open web service that will read requests you make to it and respond with various data in the response body, based on the API endpoint you request. The **/response-headers** endpoint will read the query string parameters you pass in the request and include them as properties in a JSON object response.

Write your own script that will make a request to <https://packt.live/2OE94LV>. Include two query parameters in the request, one with a key of **first** property passing **John** as the value, and another for **last** property with **Doe** as the value. Be sure to set the **Accept** header to **application/json**. Check the response for a status code of 200 and throw an exception if it does not match that. Decode the response from JSON and echo the values for the first and last properties in the decoded object in a string to output.

The output should look like this:



```
mark@WCG6Y6HYF2:~/guzzle-example$ php httpbin.php
The web service responded with John Doe
mark@WCG6Y6HYF2:~/guzzle-example$
```

Figure 10.12: The expected output

The following steps will help you to complete the activity:

1. Create a **httpbin.php** file in the **guzzle-example** directory. Require the Composer autoload file and import the **Guzzle Client** class.

2. Instantiate a new **Guzzle Client** by passing the **httpbin** address.
3. Inside a **try...catch** block, make a **POST** request to the **/response-headers** endpoint. Add an **Accept** header set to **application/json** and set two query parameter key-value pairs, with **first** as **John** and **last** as **Doe**.
4. Check whether the HTTP status code is not 200, and if so, throw an exception.
5. Parse the response body into an object using **json_decode()** and store it in a variable.
6. Output a string, **The web service responded with**, concatenated with the first and last properties from the response object.
7. Run the script and see whether the output contains **John Doe**.

Note

The solution for this activity can be found via [this link](#).

Summary

Web services are one of the most important concepts in modern-day computing, enabling many of the rich internet applications we use today. In this chapter, we have discussed some of the criteria you would want to use while evaluating web services to use in your application, such as documentation, availability, and pricing. We briefly covered the concepts of a RESTful web service, which are stateless services that expose an interface to interact with resources through the HTTP verbs. We covered the JSON and XML formats, which are hierarchical structures used to transfer data in the body of requests, among other uses.

HTTP requests are made up of a body and a number of headers, some required, some optional, and others that contain metadata about a request and negotiate the content type. We went over the authentication methods commonly utilized by web service providers, including API keys and Open ID Connect combined with OAuth 2.0 for authorization. A REST client is a useful tool to have in your toolbox to manually test API endpoints as you are working to integrate with them. Guzzle is an abstraction layer for making HTTP requests in PHP, available via the Composer package manager, that provides a clean and simple interface.